

**MountainCar-v0 con Q-Learning tabular y DQN: exploración persistente en un entorno de
recompensa plana**

Leonar Socarrás Molina

John Jairo Serrano Cifuentes

Brezhnev Joya Miranda

Gabriel Alonso Lizano Alvarado

Bryan Johann Aranzazu Medina

Maestría en Inteligencia Artificial, Universidad de La Sabana

Simulación y Aprendizaje por Refuerzo

Prof. Emilio Muñoz Pérez

20 de septiembre de 2026

U2 Taller 1. Repositorio: https://github.com/leonarsomo/mountain_car

Resumen

Comparo dos agentes de aprendizaje por refuerzo en MountainCar-v0: Q-Learning tabular sobre una rejilla de 20×20 y una Deep Q-Network (DQN) sobre la observación continua. Entrené ambos con semilla fija y los evalué en 100 episodios sin exploración, con semillas distintas de las que usé para entrenar y para escoger el punto de control. El agente tabular obtuvo una recompensa media de -123.0 (bandera en 98 de 100 episodios) y DQN de -96.7 (100 de 100), con cerca de nueve veces menos interacción con el entorno. El hallazgo de mayor interés está en la exploración. Con el epsilon-greedy de libro, DQN no aprende: en 300 episodios de acciones uniformes la bandera se alcanzó 0 veces, y una red que solo ve recompensas de -1 concluye que ninguna acción importa. Sostener la acción exploratoria durante unos 20 pasos (Dabney et al., 2020) resuelve el problema sin tocar la regla de aprendizaje.

Palabras clave: aprendizaje por refuerzo, Q-learning, DQN, exploración, recompensa plana, reproducibilidad.

MountainCar-v0 con Q-Learning tabular y DQN

Este informe documenta el Taller 1 de la Unidad 2 del curso Simulación y Aprendizaje por Refuerzo. El código, los registros de cada corrida, las figuras y los agentes entrenados están en el repositorio público https://github.com/leonarsomo/mountain_car, derivado del repositorio base del curso. Aquí reúno la formulación del problema, lo que implementé, el diagnóstico del Ejercicio 3, los dos esquemas de entrenamiento, el mejor resultado de cada agente y la comparación entre ambos.

El problema como proceso de decisión

Un carro con motor débil está en el fondo de un valle y debe llegar a la bandera de la colina derecha. El motor no vence la pendiente, así que la única salida es balancearse para ganar impulso. Sutton y Barto (2018) presentan esta tarea como un caso de control en el que las cosas deben empeorar antes de mejorar, porque el carro tiene que alejarse de la meta para alcanzarla (pp. 244-245). Esa descripción coincide con lo que observé: los dos agentes entrenados retroceden primero hacia la colina izquierda. La Tabla 1 formula el entorno como proceso de decisión de Markov con los valores que documenta la Farama Foundation (2024).

Tabla 1

Formulación de MountainCar-v0 como proceso de decisión de Markov

Elemento	Definición
Estado s	Posición x en $[-1.2, 0.6]$ y velocidad v en $[-0.07, 0.07]$, ambas continuas
Acción a	0 empujar a la izquierda, 1 no empujar, 2 empujar a la derecha
Transición	$v' = v + (a - 1) \cdot 0.001 - \cos(3x) \cdot 0.0025$; $x' = x + v'$. Es determinista
Recompensa	-1 en cada paso, sin excepción
Inicio	x uniforme en $[-0.6, -0.4]$, $v = 0$
Fin	Terminación si $x \geq 0.5$; truncamiento a los 200 pasos
Descuento	$\gamma = 0.99$

Nota. Valores tomados de Farama Foundation (2024). El descuento es el del repositorio base.

Dos consecuencias gobiernan el resto del trabajo. Como cada paso cuesta -1, el retorno del episodio es el número de pasos con signo negativo, de modo que -200 equivale a no haber llegado nunca. Además, la recompensa es plana: no distingue un paso que acerca a la bandera de uno que aleja. La única señal informativa es la terminación, y un agente que nunca la presencia no tiene de dónde aprender.

Implementación

El repositorio base trae escritos la interfaz de línea de comandos, los bucles de entrenamiento y la persistencia. Los algoritmos venían como bloques EXERCISE que lanzaban `NotImplementedError`. Completé los ejercicios 1a, 1b y 1c en `qlearning.py`, y los ejercicios 2a, 2b y 3 en `dqn.py`. Los Apéndices A a F reproducen el código completo, con las líneas que escribí sombreadas en los dos archivos que vienen del repositorio base (Muñoz Pérez, 2026).

Q-Learning tabular

Cada dimensión de la observación se parte en 20 intervalos con `np.digitize` sobre los bordes interiores, lo que da una rejilla de 400 celdas. La clave de la tabla es la tupla de índices de posición y velocidad. La tabla es un `defaultdict` que entrega tres ceros para cualquier celda no visitada, y ese cero inicial tiene un efecto que no es evidente. Sutton y Barto (2018) señalan, para este mismo problema, que un valor inicial de cero es optimista porque todas las recompensas reales son negativas, y que ese optimismo empuja al agente a explorar (p. 246). En mi corrida, una acción jamás probada siempre parecía mejor que una probada, y la tabla exploró aunque epsilon fuera pequeño. El Fragmento 1 muestra las tres piezas que escribí.

Fragmento 1

Discretización, selección de acción y actualización en Q-Learning tabular

```
# Ejercicio 1a: indices en [0, n_bins-1] con los bordes interiores
return tuple(int(np.digitize(v, edges))
              for v, edges in zip(obs, self._bins))

# Ejercicio 1b: epsilon-greedy; con deterministic=True nunca explora
if not deterministic and np.random.random() < self.epsilon:
    return int(np.random.randint(self.n_actions))
return int(np.argmax(self.q_table[state]))

# Ejercicio 1c: el bootstrap se corta con terminated, no con done
target = reward if terminated else \
    reward + self.gamma * float(np.max(self.q_table[next_state]))
self.q_table[state][action] += self.lr * (target - self.q_table[state][action])
```

Nota. Extracto de `src/mountain_car/agents/qlearning.py`. La regla de actualización es la de Sutton y Barto (2018, p. 131).

El corte del bootstrap usa `terminated` y no `done`. Agotar los 200 pasos es un límite de tiempo que impone el entorno, no un estado terminal del proceso de decisión. Tratarlo como terminal le enseñaría al agente que quedarse sin tiempo cuesta solo -1. Usé los hiperparámetros del repositorio base sin ajuste: tasa de aprendizaje 0.1, descuento 0.99 y epsilon de 1.0 a 0.01 con factor 0.9995 por episodio.

DQN

La red es un perceptrón de $2 \rightarrow 128 \rightarrow 128 \rightarrow 3$ con ReLU en las capas ocultas y salida lineal, con 17 283 parámetros. La salida no lleva activación porque son valores Q, todos negativos en esta tarea. El paso de aprendizaje toma un mini-lote de 64 transiciones al azar de la memoria de repetición, mecanismo que Mnih et al. (2013) introdujeron para romper la correlación entre muestras consecutivas. El Fragmento 2 muestra ese paso.

Fragmento 2

Paso de aprendizaje de DQN sobre un mini-lote

```
current_q = self.q_net(states_t).gather(1, actions_t) # (64, 1)
with torch.no_grad():
    next_q = self.target_net(next_states_t).max(dim=1, keepdim=True).values
    target_q = rewards_t + self.gamma * next_q * (1.0 - terminateds_t)
loss = self.loss_fn(current_q, target_q)
self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()
```

Nota. Extracto del método `_learn` en `src/mountain_car/agents/dqn.py`.

El objetivo sale de la red congelada y sin gradiente. Zhao (2025) explica la razón: el parámetro w aparece también dentro del objetivo, y suponerlo fijo durante un tiempo es lo que vuelve tratable el gradiente (pp. 182-183). Esa explicación me llevó a verificar que `current_q` y `target_q` tuvieran la misma forma (64, 1), porque una difusión silenciosa de formas entrena sobre basura sin lanzar error. Los hiperparámetros son los del repositorio base: Adam con tasa $1e-3$, memoria de 100 000 transiciones, sincronización de la red objetivo cada 10 episodios y epsilon de 1.0 a 0.01 con factor 0.995. Agregué uno, `explore_repeat = 0.95`, que explico en la sección siguiente.

Protocolo de medición

El script `scripts/experimento.py` fija la semilla 7 en random, NumPy, PyTorch y el entorno. Corta el entrenamiento en tramos de 500 episodios para la tabla y de 50 para DQN. Al cerrar cada tramo juega 20 episodios voraces sobre semillas fijas y, si la media supera a la mejor vista, guarda esa versión del agente. La cifra que reporto sale de una evaluación final aparte de 100 episodios, con semillas que no usé ni para entrenar ni para escoger el punto de control. Así la selección no infla el resultado.

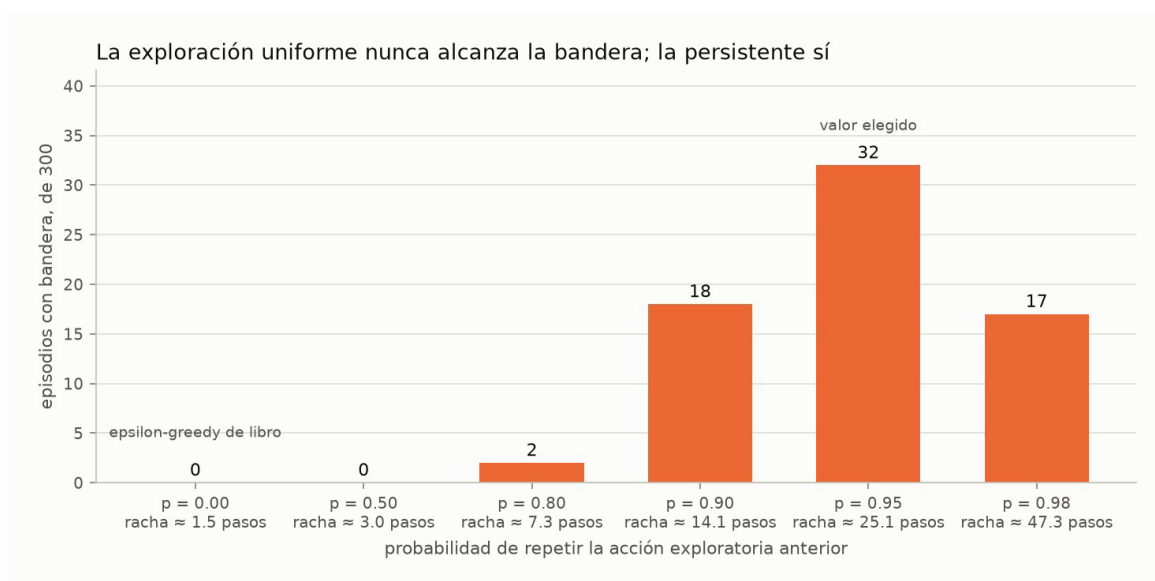
Por qué DQN no aprendía: el Ejercicio 3

Con los ejercicios 2a y 2b correctos, DQN se quedó clavado en -200. Antes de tocar nada, medí. Primero descarté el código de aprendizaje: el mismo agente, con epsilon-greedy de libro, pasa en CartPole-v1 de una recompensa media de 19.8 a 157.4 en 200 episodios. El problema era propio de MountainCar.

Después medí la exploración. El script `scripts/diagnostico_exploracion.py` juega 300 episodios con acciones al azar y cuenta cuántos terminan. Lo corrí con seis valores de la probabilidad de repetir la acción anterior. La Figura 1 resume el resultado. Con sorteo independiente en cada paso, la racha media de una misma acción es de paso y medio y la bandera se alcanzó 0 veces en 300 episodios. El carro necesita empujes sostenidos de unos veinte pasos, y la probabilidad de sacar veinte veces seguidas la misma acción entre tres es $(1/3)^{20}$, del orden de 3 en 10 000 millones.

Figura 1

Episodios que alcanzan la bandera según la persistencia de la exploración



Nota. Cada barra corresponde a 300 episodios de acciones aleatorias. Con probabilidad de repetir 0.95 se obtuvieron 32 banderas; con 0.00, ninguna. Elaboración propia a partir de `scripts/diagnostico_exploracion.py`.

Por último comprobé qué aprendía la red con esos datos. Entrené 500 episodios con `explore_repeat = 0.0`, que reproduce el epsilon-greedy de libro: 0 banderas, con 100 000 transiciones en memoria. Si todas las transiciones valen -1 y ninguna termina, todas las acciones valen lo mismo en todos los estados. La red aprendió que nada de lo que hace importa, y con esos datos tenía razón. El fallo estaba aguas arriba del aprendizaje, en la recolección de datos.

Queda por explicar por qué la tabla sí aprende con el mismo epsilon-greedy. Mi lectura es que la salvan los ceros optimistas, que la empujan de forma sistemática hacia celdas no visitadas. La red carece de ese mecanismo, porque generaliza entre estados vecinos y sus valores iniciales son arbitrarios.

La corrección

Cuando epsilon decide explorar, la acción sorteada se mantiene: en cada paso siguiente continúa con probabilidad 0.95, sin consultar la red ni volver a tirar epsilon. La racha dura en promedio $1/(1 - 0.95) = 20$ pasos. Dabney et al. (2020) atribuyen la debilidad de epsilon-greedy a su falta de persistencia temporal y proponen repetir la acción exploratoria durante una duración aleatoria. Mi versión es un caso particular de esa idea, con duración geométrica, y elegí 0.95 porque fue el valor con más banderas en el diagnóstico. El Fragmento 3 muestra el cambio.

Fragmento 3

Exploración persistente en la selección de acción de DQN

```
if not deterministic:
    # racha en curso: se repite con probabilidad explore_repeat
    if (self._last_explore_action is not None
        and random.random() < self.explore_repeat):
        return self._last_explore_action
    # nueva exploracion: se sortea la accion y se abre la racha
    if random.random() < self.epsilon:
        self._last_explore_action = random.randrange(self.action_dim)
        return self._last_explore_action
    self._last_explore_action = None
# accion voraz
with torch.no_grad():
    q = self.q_net(torch.as_tensor(state).unsqueeze(0))
return int(q.argmax(dim=1).item())
```

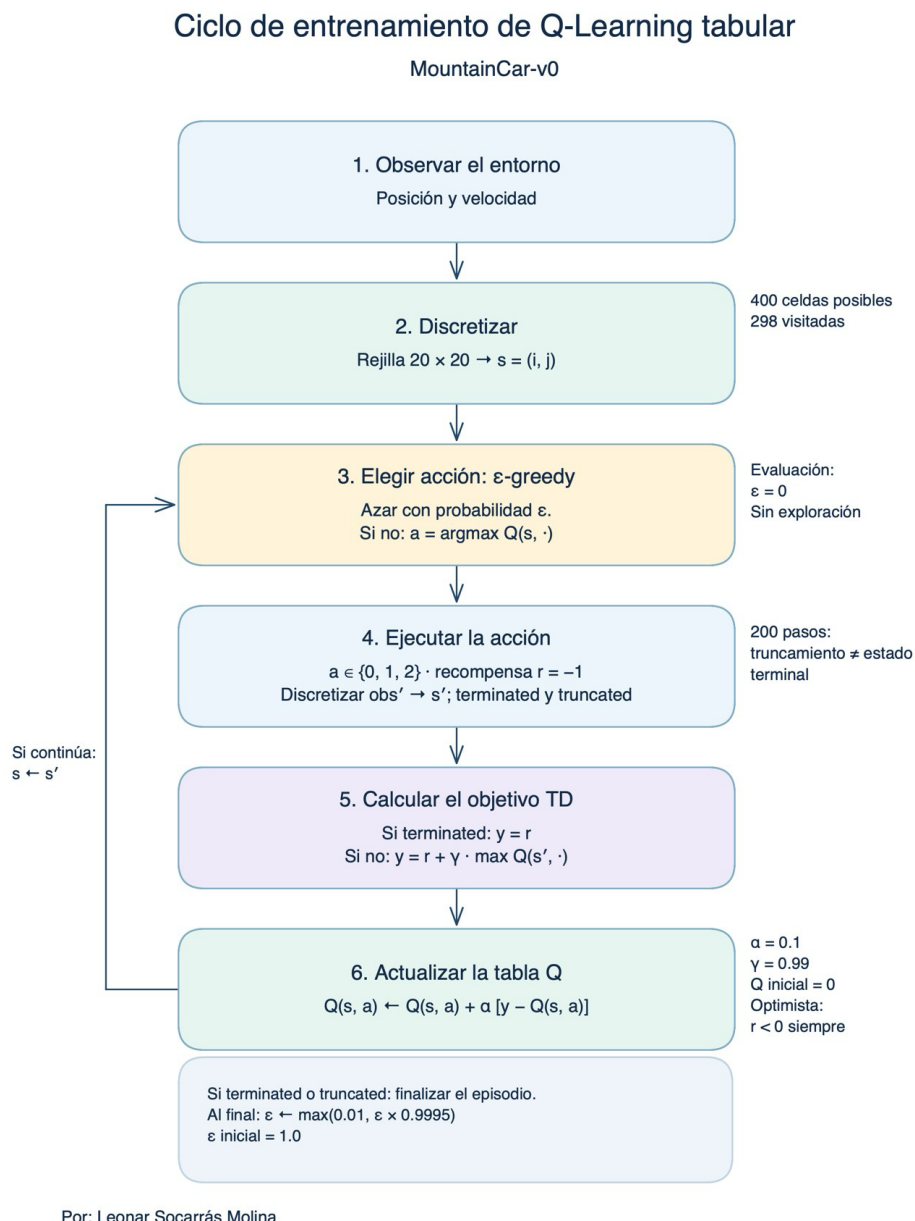
Nota. Extracto adaptado de `select_action` en `src/mountain_car/agents/dqn.py`. El estado de la racha se reinicia al comenzar cada episodio.

La corrección cambia la política de comportamiento y deja intactos la regla de aprendizaje, la recompensa y el entorno. Zhao (2025) señala como ventaja de los métodos off-policy el poder aprender la política óptima a partir de datos generados por una política muy exploratoria (p. 141). Por eso el cambio es legítimo: Q-learning aprende los valores de la política voraz aunque los datos vengan de otra. Una primera versión falló y la dejo registrada. Repetía la acción solo cuando epsilon volvía a elegir explorar; como epsilon cae rápido, las acciones

voraces cortaban la racha, y en 850 episodios no hubo una sola bandera. La versión definitiva logró 27 banderas en los primeros 300 episodios.

Esquemas de entrenamiento

Los dos esquemas son de mi autoría. Primero los dibujé a mano y después los pasé a limpio en Excalidraw. La Figura 2 muestra el ciclo de Q-Learning tabular: la observación se discretiza para obtener el estado, se elige la acción con epsilon-greedy, el entorno devuelve la recompensa y el siguiente estado, se calcula el objetivo TD y se actualiza la tabla. La flecha de la izquierda cierra el ciclo mientras el episodio continúa.

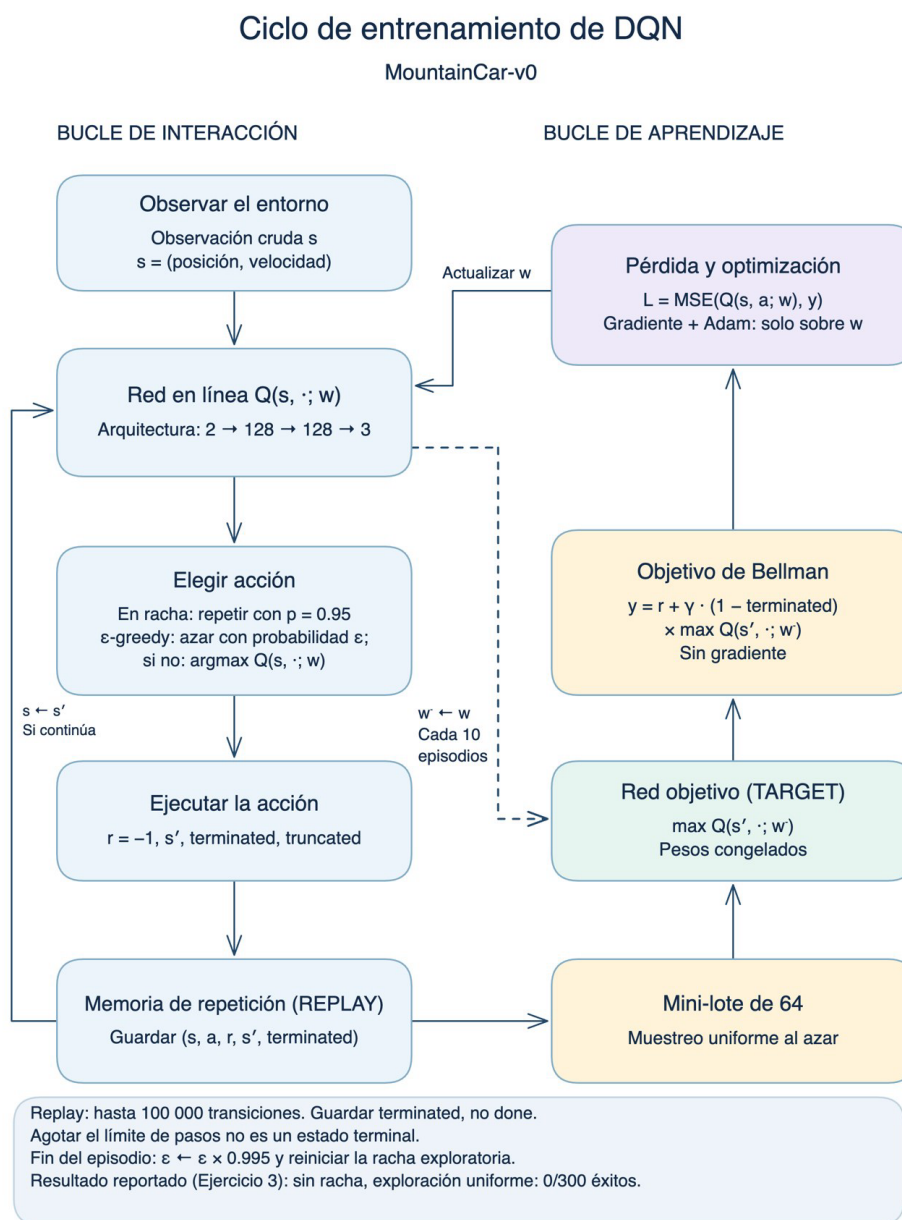
Figura 2*Ciclo de entrenamiento de Q-Learning tabular**Nota.* Elaboración propia.

La Figura 3 muestra DQN como dos bucles que se encuentran en la memoria de repetición. En el de interacción, la red en línea elige la acción y cada transición se guarda. En el de aprendizaje, un mini-lote al azar pasa por la red objetivo para formar el objetivo de Bellman,

y la pérdida actualiza solo los pesos de la red en línea. Cada 10 episodios esos pesos se copian a la red objetivo.

Figura 3

Ciclo de entrenamiento de DQN



Por: Leonar Socarrás Molina

Nota. Elaboración propia.

Resultados

La Tabla 2 reúne las cifras principales y la Figura 4 muestra las curvas de entrenamiento de ambos agentes con el mismo eje vertical.

Tabla 2

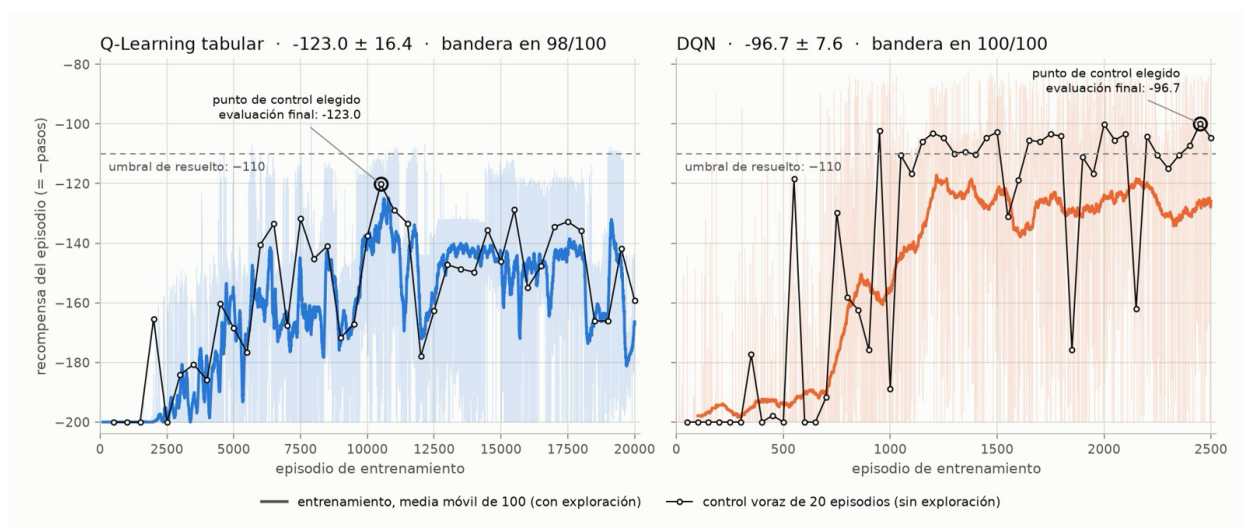
Resultados de la evaluación final de cada agente

Medida	Q-Learning tabular	DQN
Recompensa media (100 episodios voraces)	-123.0 ± 16.4	-96.7 ± 7.6
Episodios que alcanzan la bandera	98 / 100	100 / 100
Mejor y peor episodio	-110 / -200	-83 / -104
Episodios de entrenamiento	20 000	2 500
Punto de control elegido	episodio 10 500	episodio 2 450
Pasos de entorno consumidos	3 258 421	374 135
Supera el umbral de -110	No	Sí

Nota. Semilla de entrenamiento 7. La evaluación usa semillas que no intervinieron en el entrenamiento ni en la selección del punto de control. Cifras completas en resultados/metricas/ del repositorio.

Figura 4

Curvas de entrenamiento de Q-Learning tabular y de DQN



Nota. La línea gruesa es la media móvil de la recompensa de entrenamiento. La línea fina con puntos son los controles voraces de 20 episodios. La marca señala el punto de control elegido. Elaboración propia.

Mejor resultado de Q-Learning

La tabla pasa los primeros 1 620 episodios en -200. La primera bandera llega en el 1 621, cuando epsilon todavía ronda 0.44. A partir de ese episodio mejora a saltos, alcanza su mejor tramo cerca del episodio 10 500 y después no se estabiliza. En la segunda mitad del entrenamiento los controles voraces oscilan entre -120 y -178, y el agente del último episodio evalúa en -153.9, treinta puntos peor que el punto de control. Por eso guardo el mejor punto y no el último.

Veo dos causas para la oscilación. Zhao (2025) recuerda que las garantías de convergencia de Q-learning, como algoritmo de aproximación estocástica, exigen tasas de aprendizaje decrecientes (pp. 140-141). Mi tasa es constante (0.1), así que la tabla ronda el punto fijo sin asentarse. La otra causa es que la celda discretizada no es un estado de Markov: dos

observaciones distintas que caen en la misma celda tienen futuros distintos, y cerca de la cima esa diferencia decide si el carro corona o rueda de vuelta. El agente visitó 298 de las 400 celdas; las demás son combinaciones que la física del entorno no permite alcanzar.

Mejor resultado de DQN

Con la exploración persistente, la primera bandera aparece en el episodio 13. El primer control voraz con bandera llega en el episodio 350 y el primero por encima de -110 en el 950, tras unos 175 000 pasos de entorno. Incluso el peor episodio de la evaluación (-104) supera al mejor de la tabla (-110).

Dos rasgos de la curva merecen explicación. La media móvil de entrenamiento, cerca de -125, queda por debajo de los controles voraces, cerca de -105. Es el costo de mi propia corrección: con epsilon en 0.01, casi todos los episodios incluyen alguna racha exploratoria de unos veinte pasos hacia el lado equivocado. El otro rasgo son las caídas aisladas de los controles (-175.7 en el episodio 1 850), seguidas de recuperación. Sutton y Barto (2018) llaman tríada mortal a la combinación de aproximación de funciones, bootstrapping y entrenamiento off-policy, y advierten que puede producir inestabilidad (p. 264). DQN reúne los tres ingredientes. La memoria de repetición y la red objetivo amortiguan esa inestabilidad sin eliminarla, y en mi curva se ve.

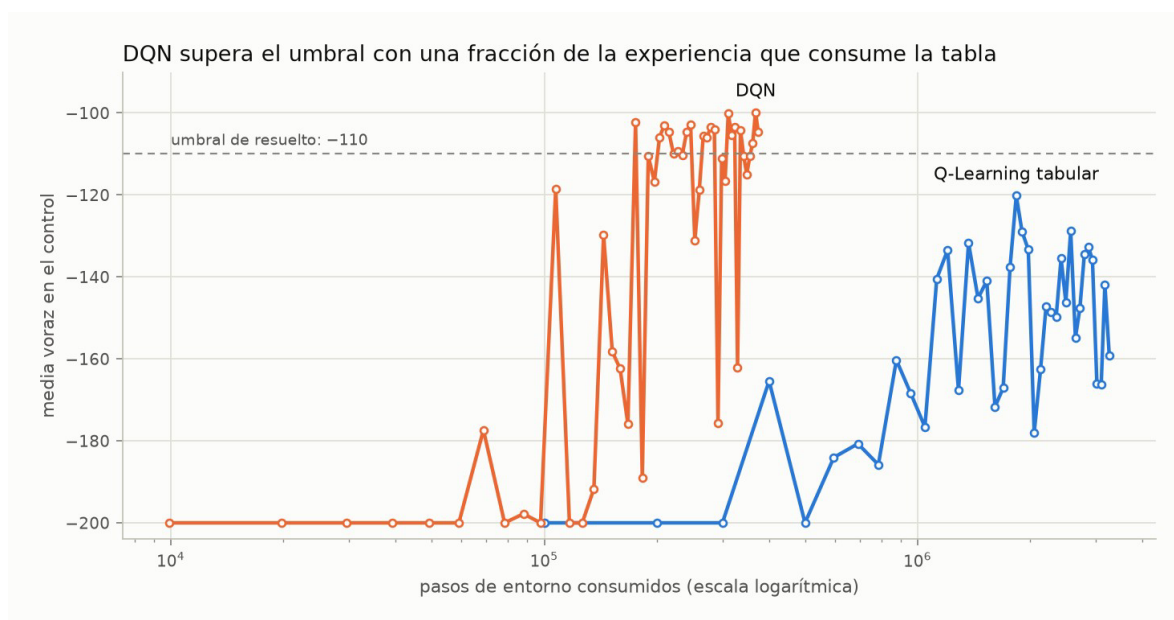
Comparación y estrategia elegida

La Tabla 3 compara los dos métodos en los criterios que pide el taller, y la Figura 5 muestra el desempeño frente a la experiencia consumida.

Tabla 3*Comparación entre Q-Learning tabular y DQN*

Criterio	Q-Learning tabular	DQN
Desempeño final	-123.0; 98/100; no supera -110	-96.7; 100/100; supera -110
Velocidad de aprendizaje	Primer control con bandera tras unos 400 000 pasos	Primer control con bandera tras unos 69 000 pasos
Tiempo de reloj	4.4 min para 20 000 episodios	13.7 min para 2 500 episodios
Estabilidad (segunda mitad)	Media -145.9; rango de -178 a -120	Media -113.3; rango de -176 a -100
Representación	400 celdas independientes	Función continua que generaliza
Dificultad de implementación	Baja: tres funciones cortas	Alta: tensores, red objetivo, memoria y exploración rediseñada
Interpretabilidad	La tabla se inspecciona celda por celda	Caja negra de 17 283 pesos

Nota. Entrenamiento en CPU de 2 núcleos con OMP_NUM_THREADS=1.

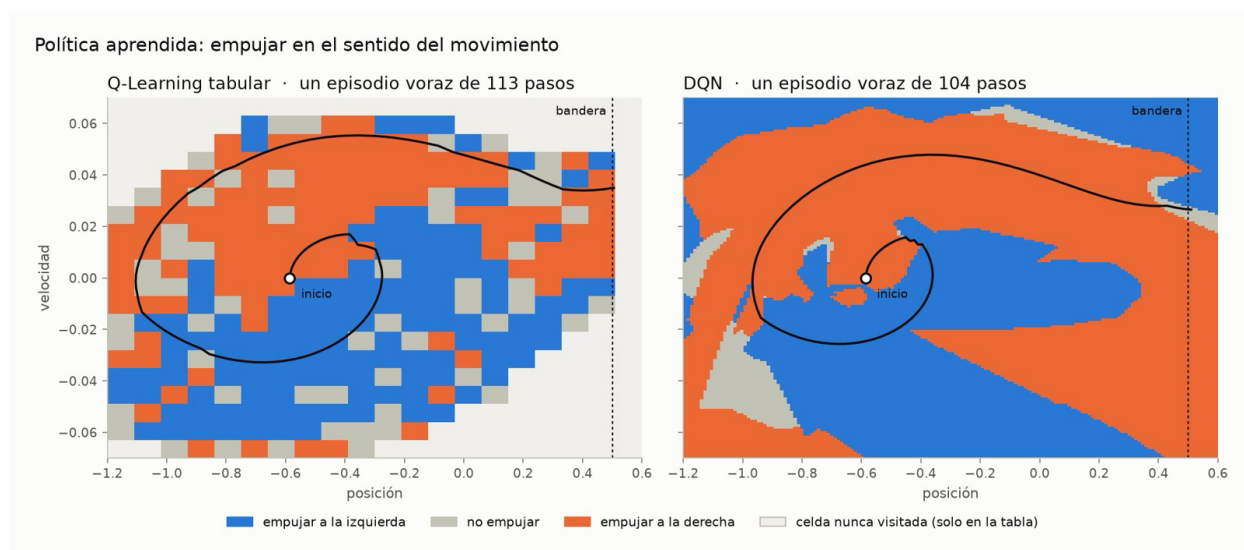
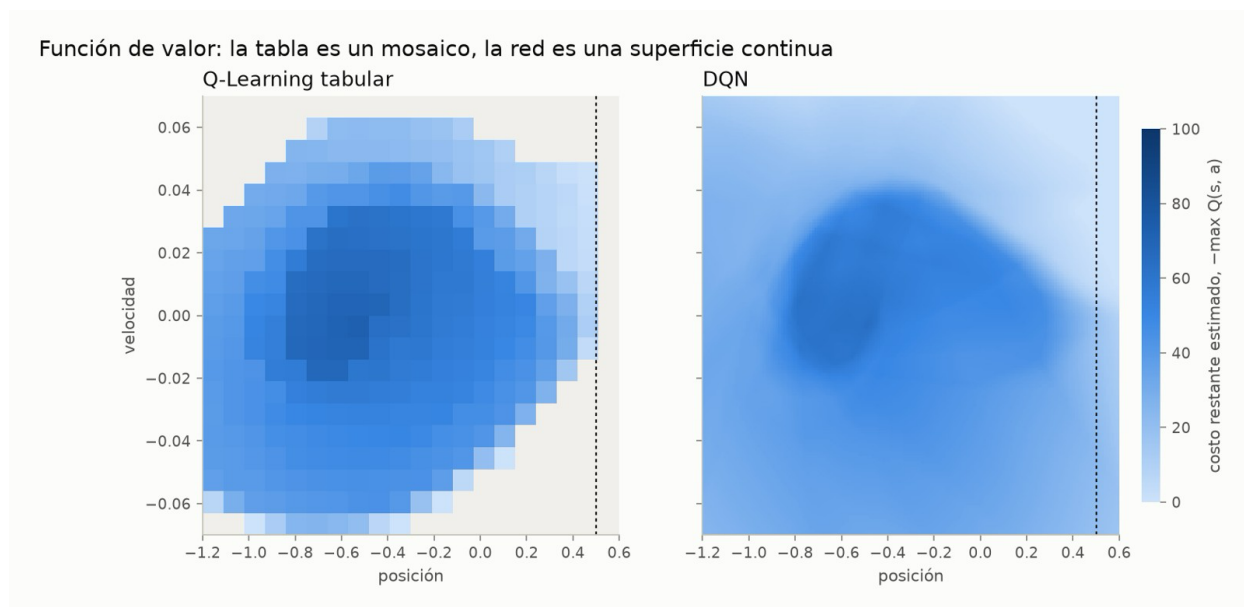
Figura 5*Desempeño voraz frente a los pasos de entorno consumidos*

Nota. Elaboración propia a partir de resultados/metricas/.

DQN aprende con cerca de nueve veces menos interacción con el entorno. Cada transición se reutiliza muchas veces desde la memoria y lo aprendido en un estado se transfiere a sus vecinos. En tiempo de reloj la relación se invierte, porque cada paso de DQN incluye un paso de gradiente y el de la tabla es una resta. Si simular fuera caro, como ocurre con un robot real, pesaría la eficiencia en muestras.

Ninguno de los dos es estable en sentido estricto. La tabla deriva de manera continua entre políticas mediocres y buenas; DQN se mantiene en una política buena y sufre caídas puntuales. Silver (2015) resume que Q-learning converge con tabla y carece de garantía con aproximación no lineal (lección 6, diap. 32). Mis resultados confirman esa teoría solo a medias, porque las condiciones de la garantía tabular (tasa decreciente, estados de Markov) no se cumplen en mi configuración.

La Figura 6 muestra qué aprendió cada agente. Los dos llegaron a la misma regla, que nadie les dio: empujar en el sentido en que el carro ya se mueve. La tabla es un mosaico con celdas sueltas; la red traza fronteras continuas y responde en todo el plano. La Figura 7 muestra el costo restante, $-\max Q(s, a)$, la misma lectura que Sutton y Barto (2018) presentan en su Figura 10.1 (p. 245). En ambos agentes el punto más caro es el fondo del valle con el carro quieto.

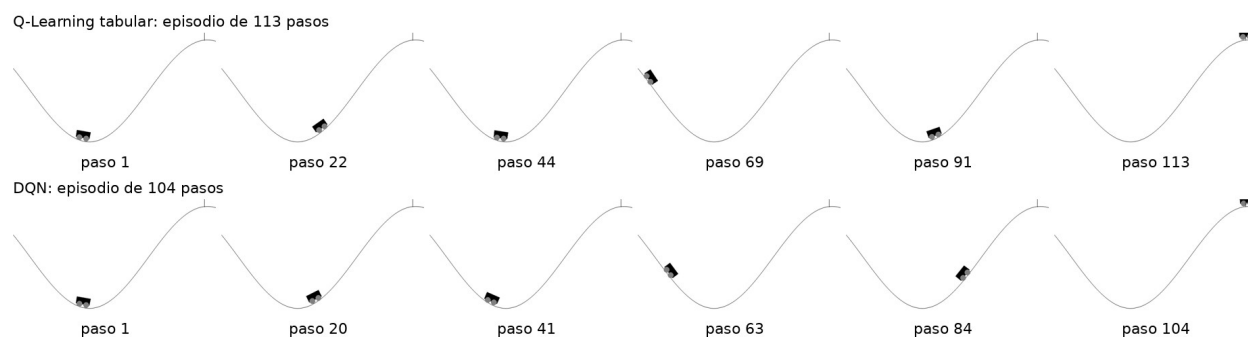
Figura 6*Acción voraz de cada agente en cada estado**Nota.* La línea superpuesta es la trayectoria de un episodio. Elaboración propia.**Figura 7***Costo restante estimado por cada agente**Nota.* Elaboración propia.

Simulación de los agentes entrenados

La Figura 8 muestra seis cuadros de un episodio voraz de cada agente, con la misma semilla de inicio. Los generé con `scripts/animar.py` (Apéndice F), que carga el punto de control guardado en `saves/`, juega el episodio con `deterministic=True` y captura el render de Gymnasium. Las animaciones completas están en el README del repositorio y cualquiera puede verlas en vivo con `uv run mountincard render dqn --episodes 3`. Ambos agentes retroceden primero hacia la colina izquierda, toman impulso y coronan; DQN lo consigue en 104 pasos y la tabla en 113.

Figura 8

Simulación de un episodio voraz de cada agente entrenado



Nota. Cuadros extraídos de `resultados/figuras/agente_qlearning.gif` y `agente_dqn.gif`. El número de paso es aproximado porque la animación guarda un cuadro cada dos pasos. Elaboración propia.

Limitaciones

El resultado principal usa una sola semilla por método. Con las semillas 1, 2 y 3, el agente tabular del último episodio evaluó en -131.0, -132.9 y -164.1, lo que pide cautela antes de generalizar. No ajusté hiperparámetros, salvo `explore_repeat`. La evaluación de DQN con varias semillas queda pendiente.

Estrategia elegida

Para MountainCar-v0 elijo DQN con exploración persistente: alcanza la bandera en todos los episodios de evaluación, es el único que supera el umbral de -110 y necesita mucha menos experiencia. El costo es una implementación más delicada y un fallo de exploración que no avisa, porque el programa corre sin errores mientras aprende que nada importa. Si el criterio fuera tener un agente funcional en diez minutos, con código que se audita a simple vista, la tabla seguiría siendo una opción razonable. Lo que me llevo del taller es que, en un problema de recompensa plana, el diseño de la exploración decide si el agente aprende.

Declaración de uso de inteligencia artificial

Usé un asistente de IA (Claude, de Anthropic) como apoyo para programar las soluciones de los ejercicios, ejecutar los entrenamientos, elaborar los scripts de medición, organizar el repositorio y redactar un borrador de este documento. Revisé el código, verifiqué las cifras contra los archivos de resultados y asumo la responsabilidad por el contenido. Los dos esquemas de entrenamiento los elaboré yo: usé guías de estudio generadas con IA para verificar el contenido y luego los dibujé por mi cuenta, primero a mano y después en Excalidraw.

Referencias

Dabney, W., Ostrovski, G., & Barreto, A. (2020). *Temporally-extended ϵ -greedy exploration*.

arXiv. <https://arxiv.org/abs/2006.01782>

Farama Foundation. (2024). *Mountain Car*. Gymnasium documentation.

https://gymnasium.farama.org/environments/classic_control/mountain_car/

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller,

M. (2013). *Playing Atari with deep reinforcement learning*. arXiv.

<https://arxiv.org/abs/1312.5602>

Muñoz Pérez, E. (2026). *mountain_car* [Software]. GitHub.

https://github.com/emiliomunozai/mountain_car

Silver, D. (2015). *Lecture 6: Value function approximation* [Diapositivas]. University College

London. <https://www.davidsilver.uk/teaching/>

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2.^a ed.). MIT

Press.

Zhao, S. (2025). *Mathematical foundations of reinforcement learning*. Springer.

<https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning>

Apéndice A

Agente de Q-Learning tabular (qlearning.py)

Archivo del repositorio base del curso (Muñoz Pérez, 2026), bajo licencia Apache 2.0, con los ejercicios 1a, 1b y 1c resueltos. Las líneas sombreadas son las que agregué o modifiqué; el resto es del repositorio base. Agregué también el parámetro opcional `seed` en `train` para fijar la semilla del entorno. Para entrenar: `uv run mountaincar train qlearning --episodes 20000`.

Archivo: `src/mountain_car/agents/qlearning.py`. 188 líneas.

```

1  """
2  Tabular Q-Learning for MountainCar-v0.
3
4  MountainCar has a 2-D continuous observation (position, velocity) with hard
5  bounds published by the environment itself, so the whole state space can be
6  discretised into a simple `n_bins x n_bins` grid -- no hand-tuned bounds and
7  no special-casing needed.
8  """
9  import pickle
10 from collections import defaultdict
11 from pathlib import Path
12 from typing import Self
13
14 import gymnasium as gym
15 import numpy as np
16
17
18 class QLearningAgent:
19     def __init__(
20         self,
21         env_id: str,
22         *,
23         n_bins: int = 20,
24         lr: float = 0.1,
25         gamma: float = 0.99,
26         epsilon_start: float = 1.0,
27         epsilon_end: float = 0.01,
28         epsilon_decay: float = 0.9995,
29     ) -> None:
30         self.env_id = env_id
31         self.n_bins = n_bins
32         self.lr = lr
33         self.gamma = gamma
34         self.epsilon = epsilon_start
35         self.epsilon_end = epsilon_end
36         self.epsilon_decay = epsilon_decay
37         self.training_episodes = 0
38
39         env = gym.make(env_id)
40         low, high = env.observation_space.low, env.observation_space.high
41         self.n_actions = int(env.action_space.n) # type: ignore[attr-defined]
42         env.close()
43
44         # Bin edges per dimension (interior edges only, as np.digitize wants).
45         self.bins = [np.linspace(lo, hi, n_bins + 1)[1:-1] for lo, hi in zip(low, high)]
46         self.q_table: dict[tuple, np.ndarray] = defaultdict(lambda: np.zeros(self.n_actions))
47
48     # — helpers —————
49
50     def discretize(self, obs: np.ndarray) -> tuple:
51         """EXERCISE 1a: map a continuous observation to a discrete table key.
52 
```

```

53     `obs` is a 2-element array (position, velocity). `self._bins[i]` holds
54     the bin edges for dimension i, already built for you in `__init__`.
55     Return a hashable key -- a tuple of one bin index per dimension.
56
57     Tip: np.digitize(value, edges) returns the index of the bin a value
58     falls into. Tip: the key must be hashable, so build a tuple of ints.
59     """
60     # np.digitize con los bordes interiores devuelve indices en [0, n_bins-1],
61     # asi que un valor fuera de rango cae en la celda extrema y no hace falta recortar.
62     return tuple(int(np.digitize(v, edges)) for v, edges in zip(obs, self._bins))
63
64     def select_action(self, state: tuple, *, deterministic: bool = False) -> int:
65         """EXERCISE 1b: epsilon-greedy action selection.
66
67         With probability `self.epsilon`, explore: pick a uniformly random
68         action in [0, self.n_actions). Otherwise exploit: pick the action with
69         the highest value in `self.q_table[state]`.
70
71         When `deterministic` is True, never explore -- always exploit. That is
72         the mode used for evaluation and rendering.
73
74         Tip: self.q_table is a defaultdict, so indexing an unseen state is safe
75         and returns a zero vector. Tip: np.argmax gives you the best action.
76         """
77         if not deterministic and np.random.random() < self.epsilon:
78             return int(np.random.randint(self.n_actions))
79         return int(np.argmax(self.q_table[state]))
80
81     def predict(self, obs: np.ndarray, *, deterministic: bool = True) -> tuple[int, None]:
82         return self.select_action(self.discretize(obs), deterministic=deterministic), None
83
84     # — core RL —————
85
86     def _update(
87         self,
88         state: tuple,
89         action: int,
90         reward: float,
91         next_state: tuple,
92         terminated: bool,
93     ) -> None:
94         """EXERCISE 1c: the Q-Learning update.
95
96         Move Q(state, action) toward the TD target:
97
98             target = reward + gamma * max_a' Q(next_state, a')
99             Q(s, a) += lr * (target - Q(s, a))
100
101         One catch: if `terminated` is True there is no next state to bootstrap
102         from, so the target is just `reward`.
103
104         Note that `terminated` is NOT the same as "the episode ended" -- see
105         the training loop below for why that distinction matters here.
106         """
107         if terminated:
108             target = reward
109         else:
110             target = reward + self.gamma * float(np.max(self.q_table[next_state]))
111             td_error = target - self.q_table[state][action]
112             self.q_table[state][action] += self.lr * td_error
113
114     def train(self, total_episodes: int = 10_000, log_interval: int = 100, seed: int | None = None) -> list[float]:
115         env = gym.make(self.env_id)
116         rewards_history: list[float] = []
117
118         for episode in range(1, total_episodes + 1):
119             # La semilla solo se pasa en el primer reset; los siguientes continuan el mismo
120             # generador.
121             obs, _ = env.reset(seed=seed if episode == 1 else None)
122             state = self.discretize(obs)
123             total_reward = 0.0
124             done = False
125
126             while not done:
127                 action = self.select_action(state)

```

```

128         next_obs, reward, terminated, truncated, _ = env.step(action)
129         done = terminated or truncated
130
131         next_state = self.discretize(next_obs)
132         self._update(state, action, float(reward), next_state, terminated)
133
134         state = next_state
135         total_reward += reward
136
137         self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)
138         self.training_episodes += 1
139         rewards_history.append(total_reward)
140
141         if episode % log_interval == 0:
142             avg = np.mean(rewards_history[-log_interval:])
143             print(
144                 f"Episode {episode}/{total_episodes} | "
145                 f"Avg Reward: {avg:.2f} | "
146                 f"Epsilon: {self.epsilon:.4f} | "
147                 f"States visited: {len(self.q_table)}"
148             )
149
150         env.close()
151         return rewards_history
152
153     # — persistence —————
154
155     _HPARAMS = ("env_id", "n_bins", "lr", "gamma", "epsilon_end", "epsilon_decay")
156
157     def save(self, path: Path) -> None:
158         path.parent.mkdir(parents=True, exist_ok=True)
159         data = {k: getattr(self, k) for k in self._HPARAMS}
160         data["q_table"] = dict(self.q_table)
161         data["epsilon"] = self.epsilon
162         data["training_episodes"] = self.training_episodes
163         with open(path, "wb") as f:
164             pickle.dump(data, f)
165         print(f"Saved Q-Learning agent to {path}")
166
167     @classmethod
168     def load(cls, path: Path) -> Self:
169         with open(path, "rb") as f:
170             data = pickle.load(f)
171
172         agent = cls(
173             data["env_id"],
174             epsilon_start=data["epsilon"],
175             **{k: data[k] for k in cls._HPARAMS if k != "env_id"},
176         )
177         agent.q_table = defaultdict(lambda: np.zeros(agent.n_actions), data["q_table"])
178         agent.training_episodes = data["training_episodes"]
179         return agent
180
181     def info(self) -> str:
182         return (
183             f"Q-Learning agent for {self.env_id}\n"
184             f" Episodes trained : {self.training_episodes}\n"
185             f" States visited   : {len(self.q_table)} / {self.n_bins ** 2}\n"
186             f" Epsilon           : {self.epsilon:.4f}\n"
187             f" LR / Gamma        : {self.lr} / {self.gamma}"
188         )

```

Apéndice B

Agente DQN (dqn.py)

Archivo del repositorio base del curso (Muñoz Pérez, 2026), con los ejercicios 2a, 2b y 3 resueltos. Las líneas sombreadas son las que agregué o modifiqué: la red, el paso de aprendizaje, la exploración persistente en `select_action`, el hiperparámetro `explore_repeat` y el reinicio de la racha en cada episodio. Para entrenar: `OMP_NUM_THREADS=1 uv run mountaincar train dqn --episodes 2500`.

Archivo: `src/mountain_car/agents/dqn.py`. 336 líneas.

```

1  """
2  Deep Q-Network (DQN) implementation in PyTorch.
3
4  This module intentionally avoids high-level RL libraries so every piece of
5  the algorithm is visible and editable for learning purposes.
6
7  Key components:
8  - QNetwork      : a small fully-connected network that maps state -> Q(s,a)
9  - ReplayBuffer  : stores (s, a, r, s', terminated) transitions for replay
10 - DQNAgent      : the training loop, epsilon-greedy policy, target-net sync
11 """
12 import random
13 from collections import deque
14 from pathlib import Path
15 from typing import Self
16
17 import gymnasium as gym
18 import numpy as np
19 import torch
20 from torch import nn, optim
21
22 # — Neural network —————
23
24
25 class QNetwork(nn.Module):
26     """EXERCISE 2a: the network that maps a state to one Q-value per action.
27
28     Build a small fully-connected net:
29
30         state_dim -> hidden -> hidden -> action_dim
31
32     with a ReLU after each hidden layer. There is NO activation on the output
33     layer: these are Q-values (here they are all negative), not probabilities.
34
35     Tip: nn.Sequential(nn.Linear(...), nn.ReLU(), ...) is the shortest route.
36     Tip: remember super().__init__() before assigning any submodule.
37     Tip: forward() receives a batch of shape (B, state_dim) and must return
38         shape (B, action_dim).
39     """
40
41     def __init__(self, state_dim: int, action_dim: int, hidden: int = 128) -> None:
42         super().__init__()
43         self.net = nn.Sequential(
44             nn.Linear(state_dim, hidden),
45             nn.ReLU(),
46             nn.Linear(hidden, hidden),
47             nn.ReLU(),
48             nn.Linear(hidden, action_dim), # salida lineal: son valores Q, no probabilidades
49         )

```

```

50
51     def forward(self, x: torch.Tensor) -> torch.Tensor:
52         return self.net(x)
53
54
55 # — Replay buffer —————
56
57
58 class ReplayBuffer:
59     """Fixed-size FIFO buffer that stores transitions for experience replay."""
60
61     def __init__(self, capacity: int = 100_000) -> None:
62         self.buffer: deque[tuple] = deque(maxlen=capacity)
63
64     def push(
65         self,
66         state: np.ndarray,
67         action: int,
68         reward: float,
69         next_state: np.ndarray,
70         terminated: bool,
71     ) -> None:
72         self.buffer.append((state, action, reward, next_state, terminated))
73
74     def sample(self, batch_size: int) -> list[tuple]:
75         return random.sample(self.buffer, batch_size)
76
77     def __len__(self) -> int:
78         return len(self.buffer)
79
80
81 # — Agent —————
82
83
84 class DQNAgent:
85     """
86     Deep Q-Network agent implemented from scratch.
87
88     Hyperparameters are intentionally exposed as constructor args so you
89     can experiment with them directly.
90     """
91
92     def __init__(
93         self,
94         env_id: str,
95         *,
96         lr: float = 1e-3,
97         gamma: float = 0.99,
98         epsilon_start: float = 1.0,
99         epsilon_end: float = 0.01,
100         epsilon_decay: float = 0.995,
101         batch_size: int = 64,
102         buffer_capacity: int = 100_000,
103         target_update_freq: int = 10,
104         hidden: int = 128,
105         explore_repeat: float = 0.95,
106     ) -> None:
107         self.env_id = env_id
108         self.lr = lr
109         self.gamma = gamma
110         self.epsilon = epsilon_start
111         self.epsilon_end = epsilon_end
112         self.epsilon_decay = epsilon_decay
113         self.batch_size = batch_size
114         self.buffer_capacity = buffer_capacity
115         self.target_update_freq = target_update_freq
116         self.hidden = hidden
117         self.explore_repeat = explore_repeat
118         self._last_explore_action: int | None = None
119         self.training_episodes = 0
120
121         env = gym.make(env_id)
122         self.state_dim = int(env.observation_space.shape[0]) # type: ignore[index]
123         self.action_dim = int(env.action_space.n) # type: ignore[attr-defined]
124         env.close()
125

```

```

126     self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
127
128     self.q_net = QNetwork(self.state_dim, self.action_dim, hidden).to(self.device)
129     self.target_net = QNetwork(self.state_dim, self.action_dim, hidden).to(self.device)
130     self.target_net.load_state_dict(self.q_net.state_dict())
131
132     self.optimizer = optim.Adam(self.q_net.parameters(), lr=lr)
133     self.loss_fn = nn.MSELoss()
134     self.buffer = ReplayBuffer(buffer_capacity)
135
136 # — policy —————
137
138 def select_action(self, state: np.ndarray, *, deterministic: bool = False) -> int:
139     """Textbook epsilon-greedy: explore with probability epsilon.
140
141     EXERCISE 3: this is the standard, by-the-book implementation, and it is
142     not enough. Once EXERCISE 2 is done, `train dqn` will run happily and
143     report a completely flat score, forever, having learned nothing.
144
145     Your job is to work out WHY and fix it. The bug is not in this method's
146     code -- it is correct epsilon-greedy. It is in what this exploration
147     strategy can actually reach in this particular environment.
148
149     Starting clue: the car is too weak to drive straight up the hill, so it
150     has to rock back and forth in sustained runs to build momentum. Every
151     call below draws a completely fresh random action. Can a policy built
152     from independent per-step coin flips produce a sustained run?
153
154     EXERCISES.md has the full investigation and a ladder of further clues,
155     from gentle to nearly-the-answer -- take only as many as you need. Try
156     to diagnose it from your own measurements first.
157     """
158     # EJERCICIO 3 (mi correccion): exploracion persistente.
159     # El sorteo uniforme paso a paso da acciones independientes; su racha media
160     # es de 1.5 pasos y en 300 episodios aleatorios la bandera se alcanza 0 veces
161     # (scripts/diagnostico_exploracion.py). El carro necesita empujes sostenidos
162     # para ganar impulso. Por eso, cuando epsilon decide explorar, la accion
163     # sorteada se mantiene: en cada paso siguiente continua con probabilidad
164     # `explore_repeat`, sin consultar la red ni volver a tirar epsilon. La racha
165     # dura en promedio 1 / (1 - explore_repeat) pasos (20 con 0.95). Es la idea
166     # del epsilon-greedy extendido en el tiempo de Dabney et al. (2020).
167     # La regla de aprendizaje, la recompensa y el entorno quedan intactos.
168     if not deterministic:
169         if self._last_explore_action is not None and random.random() < self.explore_repeat:
170             return self._last_explore_action
171         if random.random() < self.epsilon:
172             self._last_explore_action = random.randrange(self.action_dim)
173             return self._last_explore_action
174         self._last_explore_action = None
175     with torch.no_grad():
176         t = torch.as_tensor(state, dtype=torch.float32, device=self.device).unsqueeze(0)
177         return int(self.q_net(t).argmax(dim=1).item())
178
179 def predict(self, obs: np.ndarray, *, deterministic: bool = True) -> tuple[int, None]:
180     return self.select_action(obs, deterministic=deterministic), None
181
182 # — learning step —————
183
184 def _tensor(self, x, dtype=torch.float32) -> torch.Tensor:
185     return torch.as_tensor(np.array(x), dtype=dtype, device=self.device)
186
187 def _learn(self) -> float:
188     """Sample a mini-batch from the buffer and take one gradient step.
189
190     Returns the batch loss value.
191     """
192     if len(self.buffer) < self.batch_size:
193         return 0.0
194
195     batch = self.buffer.sample(self.batch_size)
196     states, actions, rewards, next_states, terminateds = zip(*batch)
197
198     states_t = self._tensor(states)
199     actions_t = self._tensor(actions, torch.int64).unsqueeze(1)
200     rewards_t = self._tensor(rewards).unsqueeze(1)
201     next_states_t = self._tensor(next_states)

```

```

202 terminated_t = self._tensor(terminateds).unsqueeze(1)
203
204 # EXERCISE 2b: the DQN learning step. Four things to do:
205 #
206 # 1. current_q : Q(s, a) from the ONLINE net, for the actions that
207 #    were actually taken. self.q_net(states_t) is (B, action_dim);
208 #    you want (B, 1). Tip: .gather(1, actions_t) picks one column
209 #    per row.
210 #
211 # 2. next_q : max_a' Q_target(s', a') from the FROZEN TARGET net.
212 #    Tip: .max(dim=1, keepdim=True).values
213 #    Tip: wrap this in `with torch.no_grad():` -- no gradient should
214 #    flow into the target, that is the whole point of a target net.
215 #
216 # 3. target_q : the Bellman target, r + gamma * next_q, but with the
217 #    bootstrap term zeroed out wherever terminateds_t is 1.
218 #    Tip: multiplying by (1.0 - terminateds_t) does this branchlessly.
219 #
220 # 4. Take one gradient step on self.loss_fn(current_q, target_q).
221 #    Tip: zero_grad() -> backward() -> step(), in that order.
222 #
223 # Return the scalar loss value (.item()).
224 current_q = self.q_net(states_t).gather(1, actions_t) # (B, 1)
225
226 with torch.no_grad():
227     next_q = self.target_net(next_states_t).max(dim=1, keepdim=True).values # (B, 1)
228     target_q = rewards_t + self.gamma * next_q * (1.0 - terminateds_t)
229
230 loss = self.loss_fn(current_q, target_q)
231 self.optimizer.zero_grad()
232 loss.backward()
233 self.optimizer.step()
234 return float(loss.item())
235
236 # — training loop —————
237
238 def train(self, total_episodes: int = 500, log_interval: int = 10, seed: int | None = None
239 ) -> list[float]:
240     env = gym.make(self.env_id)
241     rewards_history: list[float] = []
242
243     for episode in range(1, total_episodes + 1):
244         # La semilla solo se pasa en el primer reset; los siguientes continúan el mismo
generador.
245         obs, _ = env.reset(seed=seed if episode == 1 else None)
246         total_reward = 0.0
247         done = False
248         self._last_explore_action = None # estado de exploración: se reinicia por episodio
249
250         while not done:
251             action = self.select_action(obs)
252             next_obs, reward, terminated, truncated, _ = env.step(action)
253             done = terminated or truncated
254
255             # Store `terminated`, not `done`: hitting the 200-step time
256             # limit is not a real terminal state, so we must keep
257             # bootstrapping through it.
258             self.buffer.push(obs, action, float(reward), next_obs, terminated)
259             self._learn()
260
261             obs = next_obs
262             total_reward += reward
263
264         self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)
265         self.training_episodes += 1
266         rewards_history.append(total_reward)
267
268         if episode % self.target_update_freq == 0:
269             self.target_net.load_state_dict(self.q_net.state_dict())
270
271         if episode % log_interval == 0:
272             avg = np.mean(rewards_history[-log_interval:])
273             print(
274                 f"Episode {episode}/{total_episodes} | "
275                 f"Avg Reward: {avg:.2f} | "
276                 f"Epsilon: {self.epsilon:.4f} | "

```

```

277         f"Buffer: {len(self.buffer)}"
278     )
279
280     env.close()
281     return rewards_history
282
283     # — persistence —————
284
285     _HPARAMS = (
286         "env_id",
287         "lr",
288         "gamma",
289         "epsilon_end",
290         "epsilon_decay",
291         "batch_size",
292         "buffer_capacity",
293         "target_update_freq",
294         "hidden",
295         "explore_repeat",
296     )
297
298     def save(self, path: Path) -> None:
299         path.parent.mkdir(parents=True, exist_ok=True)
300         data = {k: getattr(self, k) for k in self._HPARAMS}
301         data["q_net_state"] = self.q_net.state_dict()
302         data["optimizer_state"] = self.optimizer.state_dict()
303         data["epsilon"] = self.epsilon
304         data["training_episodes"] = self.training_episodes
305         torch.save(data, path)
306         print(f"Saved DQN agent to {path}")
307
308     @classmethod
309     def load(cls, path: Path) -> Self:
310         data = torch.load(path, weights_only=False)
311         agent = cls(
312             data["env_id"],
313             epsilon_start=data["epsilon"],
314             **{k: data[k] for k in cls._HPARAMS if k != "env_id"},
315         )
316         # The target net starts as a copy of the online net; it re-syncs during
317         # training anyway, so there is no need to persist it separately.
318         agent.q_net.load_state_dict(data["q_net_state"])
319         agent.target_net.load_state_dict(data["q_net_state"])
320         agent.optimizer.load_state_dict(data["optimizer_state"])
321         agent.training_episodes = data["training_episodes"]
322         return agent
323
324     def info(self) -> str:
325         params = sum(p.numel() for p in self.q_net.parameters())
326         return (
327             f"DQN agent for {self.env_id}\n"
328             f" Episodes trained   : {self.training_episodes}\n"
329             f" Network params      : {params:,}\n"
330             f" Epsilon              : {self.epsilon:.4f}\n"
331             f" LR / Gamma          : {self.lr} / {self.gamma}\n"
332             f" Batch size          : {self.batch_size}\n"
333             f" Target update       : every {self.target_update_freq} episodes\n"
334             f" Explore repeat      : {self.explore_repeat}\n"
335             f" Device              : {self.device}"
336         )

```

Apéndice C

Diagnóstico de exploración (diagnostico_exploracion.py)

Script propio. Juega 300 episodios de acciones aleatorias por cada probabilidad de repetir la acción y cuenta cuántos alcanzan la bandera. Produce los datos de la Figura 1. Para ejecutarlo: `uv run python scripts/diagnostico_exploracion.py`.

Archivo: scripts/diagnostico_exploracion.py. 44 líneas.

```

1  """Ejercicio 3, paso 3: medir en vez de suponer.
2
3  Cuenta cuantos episodios alcanzan la bandera con dos formas de explorar:
4  - uniforme: una accion nueva e independiente en cada paso (epsilon-greedy clasico)
5  - persistente: se repite la accion anterior con probabilidad p
6  """
7  import argparse
8
9  import gymnasium as gym
10 import numpy as np
11
12
13 def contar(p_repetir: float, episodios: int, seed: int) -> tuple[int, float]:
14     rng = np.random.default_rng(seed)
15     env = gym.make("MountainCar-v0")
16     env.reset(seed=seed)
17     banderas, rachas = 0, []
18     for _ in range(episodios):
19         env.reset()
20         accion, racha, fin = int(rng.integers(3)), 1, False
21         while not fin:
22             if rng.random() >= p_repetir:
23                 nueva = int(rng.integers(3))
24                 if nueva != accion:
25                     rachas.append(racha)
26                     racha = 0
27                     accion = nueva
28             racha += 1
29             _, _, terminated, truncated, _ = env.step(accion)
30             fin = terminated or truncated
31         banderas += int(terminated)
32     env.close()
33     return banderas, float(np.mean(rachas))
34
35
36 if __name__ == "__main__":
37     ap = argparse.ArgumentParser()
38     ap.add_argument("--episodes", type=int, default=300)
39     ap.add_argument("--seed", type=int, default=7)
40     args = ap.parse_args()
41     print(f'{'p_repetir':>10} | {'banderas':>10} | racha media (pasos)')
42     for p in (0.0, 0.5, 0.8, 0.9, 0.95, 0.98):
43         b, r = contar(p, args.episodes, args.seed)
44         print(f'{'p':>10.2f} | {'b':>4}/{args.episodes:<5} | {'r':6.1f}')

```

Apéndice D

Protocolo de entrenamiento y evaluación (experimento.py)

Script propio. Fija las semillas, entrena por tramos, hace los controles voraces, guarda el mejor punto de control y corre la evaluación final de 100 episodios. Escribe los archivos de resultados/metricas/ de los que salen la Tabla 2 y las Figuras 4 y 5.

Archivo: scripts/experimento.py. 114 líneas.

```

1  """Entrena un agente con semilla fija, lo evalua por tramos y conserva el mejor.
2
3  Uso:
4      uv run python scripts/experimento.py qlearning --episodes 20000 --chunk 500
5      uv run python scripts/experimento.py dqn --episodes 2500 --chunk 50
6
7  El entrenamiento se corta en tramos de `--chunk` episodios. Al cerrar cada tramo se
8  juegan `--check-episodes` episodios voraces (sin exploracion) sobre semillas fijas,
9  y si la media supera a la mejor vista hasta entonces se guarda esa version del agente
10 en saves/, con el nombre que usa la CLI. Asi el archivo guardado corresponde al mejor
11 punto del entrenamiento y no al ultimo, que en MountainCar puede ser bastante peor.
12
13 La cifra que se reporta sale de una evaluacion final aparte, con semillas que no se
14 usaron ni para entrenar ni para elegir el punto de control.
15
16 Archivos que deja en resultados/metricas/:
17     <agente>_historial.csv      recompensa de cada episodio de entrenamiento
18     <agente>_controles.csv     media voraz al cierre de cada tramo
19     <agente>_evaluacion.json   resumen numerico
20 """
21 import argparse
22 import json
23 import random
24 import time
25 from pathlib import Path
26
27 import gymnasium as gym
28 import numpy as np
29 import torch
30
31 from mountain_car.cli import AGENTS, ENV_ID, _run_episode
32
33 OUT = Path("resultados") / "metricas"
34
35
36 def evaluar(agent, episodios: int, seed: int) -> tuple[np.ndarray, int]:
37     env = gym.make(ENV_ID)
38     env.reset(seed=seed)
39     results = [_run_episode(env, agent) for _ in range(episodios)]
40     env.close()
41     return np.array([r for r, _, _ in results]), int(sum(ok for _, _, ok in results))
42
43
44 def main() -> None:
45     ap = argparse.ArgumentParser()
46     ap.add_argument("agent", choices=tuple(AGENTS))
47     ap.add_argument("--episodes", type=int, required=True)
48     ap.add_argument("--chunk", type=int, required=True)
49     ap.add_argument("--seed", type=int, default=7)
50     ap.add_argument("--check-episodes", type=int, default=20)
51     ap.add_argument("--eval-episodes", type=int, default=100)
52     args = ap.parse_args()
53
54     random.seed(args.seed)
55     np.random.seed(args.seed)
56     torch.manual_seed(args.seed)

```

```

57
58     cls, save_path = AGENTS[args.agent]
59     agent = cls(ENV_ID)
60
61     history: list[float] = []
62     controles: list[tuple[int, float, int]] = []
63     best_mean, best_at = -np.inf, 0
64     t0 = time.perf_counter()
65
66     for start in range(0, args.episodes, args.chunk):
67         n = min(args.chunk, args.episodes - start)
68         history += agent.train(total_episodes=n, seed=args.seed if start == 0 else None)
69         rewards, flags = evaluar(agent, args.check_episodes, seed=args.seed + 500)
70         controles.append((start + n, float(rewards.mean()), flags))
71         if rewards.mean() > best_mean:
72             best_mean, best_at = float(rewards.mean()), start + n
73             agent.save(save_path)
74         print(f" control @ {start + n}: media voraz {rewards.mean():.2f}, banderas
{flags}/{args.check_episodes}")
75
76     minutes = (time.perf_counter() - t0) / 60
77     final_rewards, final_flags = evaluar(agent, args.eval_episodes, seed=args.seed + 1000)
78
79     best = cls.load(save_path)
80     rewards, flags = evaluar(best, args.eval_episodes, seed=args.seed + 1000)
81
82     OUT.mkdir(parents=True, exist_ok=True)
83     np.savetxt(OUT / f"{args.agent}_historial.csv", history, fmt="%.0f", header="recompensa",
comments="")
84     np.savetxt(
85         OUT / f"{args.agent}_controles.csv", controles, fmt=["%d", "%.2f", "%d"],
86         delimiter=",", header="episodio,media_voraz,banderas", comments="",
87     )
88
89     hist = np.array(history)
90     moving = np.convolve(hist, np.ones(100) / 100, mode="valid")
91     summary = {
92         "agente": args.agent,
93         "semilla": args.seed,
94         "episodios_entrenamiento": args.episodes,
95         "minutos_entrenamiento": round(minutes, 2),
96         "primer_episodio_con_bandera": int(np.argmax(hist > -200)) + 1 if (hist > -200).any() else
None,
97         "mejor_episodio_entrenamiento": float(hist.max()),
98         "mejor_media_movil_100_entrenamiento": round(float(moving.max()), 2),
99         "punto_de_control_elegido": best_at,
100        "eval_episodios": args.eval_episodes,
101        "eval_media": round(float(rewards.mean()), 2),
102        "eval_desviacion": round(float(rewards.std()), 2),
103        "eval_mejor": float(rewards.max()),
104        "eval_peor": float(rewards.min()),
105        "eval_banderas": flags,
106        "eval_media_ultimo_modelo": round(float(final_rewards.mean()), 2),
107        "eval_banderas_ultimo_modelo": final_flags,
108    }
109    (OUT / f"{args.agent}_evaluacion.json").write_text(json.dumps(summary, indent=2) + "\n")
110    print(json.dumps(summary, indent=2))
111
112
113 if __name__ == "__main__":
114     main()

```

Apéndice E

Pruebas automáticas (test_agentes.py)

Archivo propio con 11 pruebas rápidas que corren en la integración continua del repositorio: formas de los tensores, corte del bootstrap con terminated, comportamiento voraz con deterministic=True y persistencia de la exploración. Para ejecutarlas: `uv run pytest -q`.

Archivo: tests/test_agentes.py. 125 líneas.

```

1  """Pruebas rapidas de los dos agentes. Corren en segundos: no entrenan hasta converger."""
2  from itertools import pairwise
3  from pathlib import Path
4
5  import gymnasium as gym
6  import numpy as np
7  import pytest
8  import torch
9
10 from mountain_car.agents import DQNAgent, QLearningAgent
11 from mountain_car.cli import _run_episode
12
13 ENV_ID = "MountainCar-v0"
14 SAVES = Path(__file__).resolve().parents[1] / "saves"
15
16
17 # — Q-Learning tabular —————
18
19
20 def test_discretize_da_indices_validos_y_hashables():
21     agent = QLearningAgent(ENV_ID, n_bins=20)
22     for obs in ([-1.2, -0.07], [0.6, 0.07], [-0.5, 0.0], [5.0, -5.0]):
23         key = agent.discretize(np.array(obs, dtype=np.float32))
24         assert isinstance(key, tuple) and len(key) == 2
25         assert all(isinstance(i, int) and 0 <= i < 20 for i in key)
26         hash(key)
27
28
29 def test_discretize_separa_observaciones_lejanas():
30     agent = QLearningAgent(ENV_ID)
31     assert agent.discretize(np.array([-1.0, -0.05])) != agent.discretize(np.array([0.4, 0.05]))
32
33
34 def test_modo_determinista_nunca_explora():
35     agent = QLearningAgent(ENV_ID, epsilon_start=1.0)
36     state = (3, 4)
37     agent.q_table[state] = np.array([-5.0, -1.0, -9.0])
38     assert {agent.select_action(state, deterministic=True) for _ in range(200)} == {1}
39
40
41 def test_actualizacion_td_coincide_con_la_formula():
42     agent = QLearningAgent(ENV_ID, lr=0.5, gamma=0.9)
43     s, s2 = (0, 0), (1, 1)
44     agent.q_table[s2] = np.array([-2.0, -4.0, -3.0])
45     agent.update(s, 2, -1.0, s2, terminated=False)
46     assert agent.q_table[s][2] == pytest.approx(0.5 * (-1.0 + 0.9 * -2.0))
47
48
49 def test_estado_terminal_no_hace_bootstrap():
50     agent = QLearningAgent(ENV_ID, lr=1.0)
51     s, s2 = (0, 0), (1, 1)
52     agent.q_table[s2] = np.array([-50.0, -50.0, -50.0])
53     agent.update(s, 0, -1.0, s2, terminated=True)
54     assert agent.q_table[s][0] == pytest.approx(-1.0)
55
56

```

```

57 # — DQN —————
58
59
60 def test_red_q_devuelve_un_valor_por_accion():
61     agent = DQNAgent(ENV_ID, hidden=16)
62     out = agent.q_net(torch.zeros(7, agent.state_dim))
63     assert out.shape == (7, agent.action_dim)
64
65
66 def test_paso_de_aprendizaje_cambia_solo_la_red_en_linea():
67     torch.manual_seed(0)
68     agent = DQNAgent(ENV_ID, hidden=16, batch_size=8)
69     env = gym.make(ENV_ID)
70     obs, _ = env.reset(seed=0)
71     for _ in range(32):
72         a = env.action_space.sample()
73         nxt, r, term, _trunc, _ = env.step(a)
74         agent.buffer.push(obs, a, float(r), nxt, term)
75         obs = nxt
76     env.close()
77     antes_linea = [p.clone() for p in agent.q_net.parameters()]
78     antes_objetivo = [p.clone() for p in agent.target_net.parameters()]
79     loss = agent._learn()
80     assert np.isfinite(loss) and loss > 0
81     assert any(not torch.equal(a, b) for a, b in zip(antes_linea, agent.q_net.parameters()))
82     assert all(torch.equal(a, b) for a, b in zip(antes_objetivo, agent.target_net.parameters()))
83
84
85 def test_exploracion_persistente_produce_rachas_largas():
86     import random
87
88     random.seed(0)
89     obs = np.zeros(2, dtype=np.float32)
90
91     def racha_medio(p: float) -> float:
92         agent = DQNAgent(ENV_ID, hidden=8, explore_repeat=p, epsilon_start=1.0)
93         acciones = [agent.select_action(obs) for _ in range(4000)]
94         cambios = sum(a != b for a, b in pairwise(acciones))
95         return len(acciones) / (cambios + 1)
96
97     assert racha_medio(0.0) < 2.0 # epsilon-greedy de libro: ~1.5 pasos
98     assert racha_medio(0.95) > 15.0 # correccion del Ejercicio 3: ~25-30 pasos
99
100
101 def test_explore_repeat_sobrevive_a_guardar_y_cargar(tmp_path):
102     agent = DQNAgent(ENV_ID, hidden=8, explore_repeat=0.8)
103     path = tmp_path / "dqn.pt"
104     agent.save(path)
105     assert DQNAgent.load(path).explore_repeat == 0.8
106
107
108 # — agentes entrenados que acompañan al repositorio —————
109
110
111 @pytest.mark.parametrize(
112     ("cls", "archivo", "umbral"),
113     [(QLearningAgent, "qlearning_mountaincar.pkl", -150.0), (DQNAgent, "dqn_mountaincar.pt", -
114         110.0)],
115 )
116
117 def test_agentes_guardados_alcanzan_la_bandera(cls, archivo, umbral):
118     path = SAVES / archivo
119     if not path.exists():
120         pytest.skip(f"no hay agente guardado en {path}")
121     agent = cls.load(path)
122     env = gym.make(ENV_ID)
123     env.reset(seed=1007)
124     resultados = [_run_episode(env, agent) for _ in range(20)]
125     env.close()
126     media = np.mean([r for r, _, _ in resultados])
127     assert media > umbral, f"media voraz {media:.1f} por debajo de {umbral}"

```

Apéndice F

Simulación de los agentes (animar.py)

Script propio. Carga cada agente guardado, juega un episodio voraz con semilla fija, captura los cuadros del render de Gymnasium y escribe las animaciones de resultados/figuras/ de las que sale la Figura 8. Para ejecutarlo: `uv run --with pillow python scripts/animar.py`.

Archivo: `scripts/animar.py`. 56 líneas.

```

1  """Graba un episodio voraz de cada agente como GIF.
2
3      uv run --with pillow python scripts/animar.py
4
5  Usa el render `rgb_array` de Gymnasium, asi que no abre ninguna ventana.
6  """
7  import os
8  from pathlib import Path
9
10 os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
11
12 import gymnasium as gym
13 import torch
14 from PIL import Image, ImageDraw
15
16 from mountain_car.agents import DQNAgent, QLearningAgent
17
18 FIG = Path("resultados") / "figuras"
19 ENV_ID = "MountainCar-v0"
20 AGENTES = {
21     "qlearning": (QLearningAgent, Path("save/qlearning_mountaincar.pkl"), "Q-Learning tabular"),
22     "dqn": (DQNAgent, Path("save/dqn_mountaincar.pt"), "DQN"),
23 }
24 NOMBRE_ACCION = {0: "<< empuja a la izquierda", 1: "-- no empuja", 2: "empuja a la derecha >>"}
25
26
27 def grabar(nombre: str, seed: int = 1007) -> int:
28     cls, path, titulo = AGENTES[nombre]
29     agente = cls.load(path)
30     env = gym.make(ENV_ID, render_mode="rgb_array")
31     obs, _ = env.reset(seed=seed)
32     cuadros, paso, fin = [], 0, False
33     while not fin:
34         accion, _ = agente.predict(obs, deterministic=True)
35         obs, _, terminated, truncated, _ = env.step(int(accion))
36         paso += 1
37         fin = terminated or truncated
38         if paso % 2 and not fin:
39             continue # un cuadro de cada dos: el GIF pesa la mitad
40         img = Image.fromarray(env.render()).resize((480, 320))
41         d = ImageDraw.Draw(img)
42         d.text((10, 8), f"{titulo} | paso {paso} | recompensa {-paso}", fill=(20, 20, 20))
43         d.text((10, 24), NOMBRE_ACCION[int(accion)], fill=(20, 20, 20))
44         cuadros.append(img.quantize(64))
45     env.close()
46     cuadros += [cuadros[-1]] * 12 # pausa al llegar
47     cuadros[0].save(FIG / f"agente_{nombre}.gif", save_all=True, append_images=cuadros[1:],
48 duration=50,
49                 loop=0, optimize=True)
50     return paso
51
52 if __name__ == "__main__":
53     torch.set_num_threads(1)
54     FIG.mkdir(parents=True, exist_ok=True)
55     for nombre in AGENTES:

```

```
56         print(f"{nombre}: episodio de {grabar(nombre)} pasos ->  
resultados/figuras/agente_{nombre}.gif")
```